UNIVERSIDADE FEDERAL DO CEARÁ

# Relatório Final

**Disciplina:** Qualidade de Software
**Equipe:** Paulo Sergio Vieira de Oliveira (566589) e Randson da Silva Alves (568002)
**Projeto:** responder (https://github.com/kennethreitz/responder)
**Fork:** https://github.com/Randson-Silva/responder
**Pull Request:** https://github.com/kennethreitz/responder/pull/629
**Commit SHA:** 17f67fd7be8dfb2735b335028d8e9e1025bcf1b8

***Observações sobre o projeto:*** *O repositório original recebeu uma atualização na sua branch main enquanto a equipe realizava as refatorações. Para lidar com isso, foi necessário criar uma branch unificadora com todas as refatorações (mantendo as 4 branches temáticas isoladas no fork) para sincronizar o projeto com as alterações da main antes de abrir o Pull Request. Além disso, o proprietário do projeto original realizou uma revisão de código e solicitou alterações, as quais foram prontamente atendidas e implementadas pela equipe.*

## 1. Apresentação do projeto

O Responder é um framework web para Python desenhado para criação de APIs de forma amigável. Os testes estão organizados em módulos padrão utilizando as ferramentas pytest para execução, radon para complexidade e TEMPY para métricas de test smells.

## 2. Estado inicial do projeto

### 2.1 Test smells detectados antes da refatoração (TEMPY)

| Test smell | Ocorrências detectadas |
|---|---|
| Unknown Test | 88 |
| Conditional Test Logic | 67 |
| Exception Handling | 20 |
| Non-Functional Statement | 17 |
| Sleepy Test | 15 |

| Test smell | Ocorrências detectadas |
|---|---|
| Programming Paradigms Blend | 11 |
| Verbose Test | 9 |
| Undefined Test | 8 |
| **Total** | 235 |

### 2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

| Métrica | Valor |
|---|---|
| Testes aprovados (Passed) | 1550 |
| Testes com falha (Failed) | 0 |
| Erros (Errors) | 2 |
| Ignorados (Skipped) | 4 |
| Cobertura total (%) | 90,3 |

### 2.3 Métricas do código de teste antes da refatoração (Radon)

| Métrica | Valor |
|---|---|
| SLOC total dos testes (linha TOTAL com origem = teste) | 17238 |
| Complexidade ciclomática média das funções de teste | 2,071 |
| Índice de manutenibilidade médio dos arquivos de teste | 47,96 |

## 3. Refatorações realizadas

### 3.1 Seleção das ocorrências

Para a nossa metade, foram selecionados os test smells **Conditional Test Logic** (10 ocorrências) e **Unknown Test** (10 ocorrências). O *Conditional Test Logic* foi escolhido com base nos impactos apontados pelo CATSPY, para remover laços iterativos e lógicas imperativas que dificultavam os tracebacks de erro, substituindo por um paradigma funcional mais previsível. Já o *Unknown Test* foi priorizado para tornar explícitas as asserções de captura de exceções, facilitando a análise estática e a clareza semântica.

### 3.2 Lista das refatorações

| ID | Test smell | Arquivo::teste | Commit | Testes passaram? | Removido pelo TEMPY? | Introduziu novo smell? |
|---|---|---|---|---|---|---|
| R01 | Conditional Test Logic | tests/test_auth.py::test_auth_injected_model_names_are_not_treated_as_request_body_models | b5bbd0c1940ebc6cd0bb3acf6496eb0f2b8a525f | sim | sim | não |
| R02 | Conditional Test Logic | tests/test_clientgen.py::test_generated_streaming_clients_pass_native_syntax_check | d696ef94c4b6f79749e45b543e9e11d06f54cb45 | sim | sim | não |
| R03 | Conditional Test Logic | tests/test_ext_810.py::test_label_values_are_escaped | bb071435de57a76998c10189bb943d49d2585387 | sim | sim | não |
| R04 | Conditional Test Logic | tests/test_auth_jwt.py::test_jwt_verify_callback_maps_claims_to_principal | 1e444f494915252b444167928de400e8a0773814 | sim | sim | não |
| R05 | Conditional Test Logic | tests/test_auth_jwt.py::test_jwt_jwks_lookup_does_not_block_event_loop | ef21bb20726441ebdff07aee44c82c5cee114d2b | sim | sim | não |
| R06 | Conditional Test Logic | tests/test_auth_jwt.py::test_router_reinclusion_with_fresh_identical_jwtauth_objects | c587e17c3844c9a5fa659bc8c85cfa60354a785e | sim | sim | não |

| ID | Test smell | Arquivo::test e | Commit | Testes passaram? | Removido pelo TEMPY? | Introduziu novo smell? |
|---|---|---|---|---|---|---|
| R07 | Conditional Test Logic | tests/test_auth_jwt.py::test_jwt_unknown_kid_flood_is_rate_limited | ee3531ce09e074673567635a270d2925b4b599fe | sim | sim | não |
| R08 | Conditional Test Logic | tests/test_cli.py::test_cli_build_invalid_package_json | 42c084f53acd6d5988c35fcc7b009ffd92ecda37 | sim | sim | não |
| R09 | Conditional Test Logic | tests/test_cli.py::test_cli_run | 1a4de327b138d00e80cbb3406dc51917f3ea69f8 | sim | sim | não |
| R10 | Conditional Test Logic | tests/test_clientgen.py::test_generate_client_supports_other_languages | 1bb0ae03787b9b9d4658e15cc27bb21436d40b5d | sim | sim | não |
| R11 | Unknown Test | tests/test_auth.py::test_bearer_requires_verify_or_tokens | f1c32d45ca250cf237908005aa824258f8d39d98 | sim | sim | não |
| R12 | Unknown Test | tests/test_auth.py::test_named_auth_policy_rejects_duplicate_names | 84575613d84e7436ac4d6d9fdf8722350212b818 | sim | sim | não |
| R13 | Unknown Test | tests/test_auth.py::test_add_security_scheme_requires_openapi | 11b90198e097acadf2da833e7412f5b25afc67d4 | sim | sim | não |

| ID | Test smell | Arquivo::teste | Commit | Testes passaram? | Removido pelo TEMPY? | Introduziu novo smell? |
|---|---|---|---|---|---|---|
| R14 | Unknown Test | tests/test_auth_jwt.py::test_jwt_requires_secret_or_jwks_url | 26dca4c47ed43d77edaf26fb21ac5a9446a828dd | sim | sim | não |
| R15 | Unknown Test | tests/test_auth_jwt.py::test_oauth2_requires_flows_and_exactly_one_validator | f046a7e13f61df411676e1e426ec6e24e64c261e | sim | sim | não |
| R16 | Unknown Test | tests/test_auth_jwt.py::test_jwt_rejects_insecure_http_jwks_url | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R17 | Unknown Test | tests/test_csrf_proxy_90.py::test_csrf_requires_sessions | cb8e652eec3d228c56c1c6137d45fca29c56f1ca | sim | sim | não |
| R18 | Unknown Test | tests/test_deprecations_810.py::test_json_decimal_rejects_unknown_mode | 9942e6d83554abffdfd8fa42f9f59c8c05d874ec | sim | sim | não |
| R19 | Unknown Test | tests/test_ext_810.py::test_buckets_must_be_ascending_and_nonempty | e4635e550f72d346ba0dde396a72f97f9f2257fc | sim | sim | não |
| R20 | Unknown Test | tests/test_ext_810.py::test_json_session_serializer_rejects_unknown_types | c621a04e5fb4403c5f949f1ccc5bf9c961240f6c | sim | sim | não |

| ID | Test smell | Arquivo::test e | Commit | Testes passaram? | Removido pelo TEMPY? | Introduziu novo smell? |
|---|---|---|---|---|---|---|
| R21 | Exception Handling | tests/test_cli.py::test_cli_version | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R22 | Exception Handling | tests/test_cli.py::test_cli_run | a8179ec7d67387d33a3c523cbe87f95be6c32edc | sim | sim | não |
| R23 | Exception Handling | tests/test_coverage.py::test_background_task_exception | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R24 | Exception Handling | tests/test_devex_810.py::test_iter_sse_endless_stream_with_heartbeat | 7864ab5b4dd041d8bc6cce3918757a66e376c1ed | sim | sim | não |
| R25 | Exception Handling | tests/test_dispatch_fixes_80.py::test_lifespan_startup_event_failure_reports_startup_failed | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R26 | Exception Handling | tests/test_plumbing_801.py::test_load_target_basic_registers_sys_modules | 90df427b379f848bf5153f74a51fda3c2d64aeba | sim | sim | não |
| R27 | Exception Handling | tests/test_plumbing_801.py::test_load_target_basic_does_not_clo | 90df427b379f848bf5153f74a51fda3c2d64aeba | sim | sim | não |

| ID | Test smell | Arquivo::test e | Commit | Testes passaram? | Removido pelo TEMPY? | Introduziu novo smell? |
|---|---|---|---|---|---|---|
| | | bber_existing_module | | | | |
| R28 | Exception Handling | tests/test_plumbing_801.py::test_load_target_basic_failure_keeps_existing_module | 90df427b379f848bf5153f74a51fda3c2d64aeba | sim | sim | não |
| R29 | Exception Handling | tests/test_plumbing_801.py::test_responder_server_constructs_off_main_thread | 90df427b379f848bf5153f74a51fda3c2d64aeba | sim | sim | não |
| R30 | Exception Handling | tests/test_plumbing_801.py::test_responder_server_signal_handlers_install_and_restore | 90df427b379f848bf5153f74a51fda3c2d64aeba | sim | sim | não |
| R31 | Sleepy Test | tests/test_auth_jwt.py::test_jwt_jwks_lookup_does_not_block_event_loop | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R32 | Sleepy Test | tests/test_cli.py::test_cli_run | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R33 | Sleepy Test | tests/test_coverage.py::test_background_task_exception | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |

| ID | Test smell | Arquivo::teste | Commit | Testes passaram? | Removido pelo TEMPY? | Introduziu novo smell? |
|---|---|---|---|---|---|---|
| R34 | Sleepy Test | tests/test_coverage.py::test_background_run | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R35 | Sleepy Test | tests/test_dispatch_fixes_80.py::test_background_tasks_drain_on_shutdown_with_lifespan | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R36 | Sleepy Test | tests/test_redis_backends.py::test_redis_session_expiry | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R37 | Sleepy Test | tests/test_redis_backends.py::test_redis_ratelimit_window_reset | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R38 | Sleepy Test | tests/test_responder.py::test_background | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R39 | Sleepy Test | tests/test_routes_801.py::test_timeout_504_uses_fresh_response | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |
| R40 | Sleepy Test | tests/test_routes_801.py::test_timeout_504_uses_fresh_response | 359ebab5957fc373c2f1473503840f00ab89e6f6 | sim | sim | não |

### 3.3 Exemplos detalhados

**R01 – Conditional Test Logic (tests/test_auth.py::test_auth_injected_model_names_are_not_treated_as_request_body_models)**

**Antes:**

```
def test_auth_injected_model_names_are_not_treated_as_request_body_models():
    class Principal(BaseModel):
        name: str

    class ItemIn(BaseModel):
        name: str

    def verify(token):
        if token == "ok":
            return Principal(name="Ada")
        return None

    api = _api()
    auth = BearerAuth(verify=verify)

    @api.post("/by-user", auth=auth)
    def by_user(req, resp, *, user: Principal, item: ItemIn):
        resp.media = {"principal": user.name, "item": item.name}

    @api.post("/by-principal", auth=auth)
    def by_principal(req, resp, *, principal: Principal, item: ItemIn):
        resp.media = {"principal": principal.name, "item": item.name}

    @api.post("/by-auth", auth=auth)
    def by_auth(req, resp, *, auth: Principal, item: ItemIn):
        resp.media = {"principal": auth.name, "item": item.name}

    client = _client(api)
    headers = {"Authorization": "Bearer ok"}
    body = {"name": "Night Market"}

    assert client.post("/by-user", json=body, headers=headers).json() == {
        "principal": "Ada",
        "item": "Night Market",
    }
    assert client.post("/by-principal", json=body, headers=headers).json() == {
        "principal": "Ada",
        "item": "Night Market",
    }
    assert client.post("/by-auth", json=body, headers=headers).json() == {
        "principal": "Ada",
        "item": "Night Market",
    }

    spec = yaml.safe_load(client.get("/schema.yml").content)
    for path in ("/by-user", "/by-principal", "/by-auth"):
        assert spec["paths"][path]["post"]["requestBody"]["content"][
            "application/json"
        ]["schema"] == {"$ref": "#/components/schemas/ItemIn"}
```

**Depois:**



```
def test_auth_injected_model_names_are_not_treated_as_request_body_models():
    class Principal(BaseModel):
        name: str

    class ItemIn(BaseModel):
        name: str

    def verify(token):
        return Principal(name="Ada")



    api = _api()
    auth = BearerAuth(verify=verify)

    @api.post("/by-user", auth=auth)
    def by_user(req, resp, *, user: Principal, item: ItemIn):
        resp.media = {"principal": user.name, "item": item.name}

    @api.post("/by-principal", auth=auth)
    def by_principal(req, resp, *, principal: Principal, item: ItemIn):
        resp.media = {"principal": principal.name, "item": item.name}

    @api.post("/by-auth", auth=auth)
    def by_auth(req, resp, *, auth: Principal, item: ItemIn):
        resp.media = {"principal": auth.name, "item": item.name}

    client = _client(api)
    headers = {"Authorization": "Bearer ok"}
    body = {"name": "Night Market"}

    assert client.post("/by-user", json=body, headers=headers).json() == {
        "principal": "Ada",
        "item": "Night Market",
    }
    assert client.post("/by-principal", json=body, headers=headers).json() == {
        "principal": "Ada",
        "item": "Night Market",
    }
    assert client.post("/by-auth", json=body, headers=headers).json() == {
        "principal": "Ada",
        "item": "Night Market",
    }

    spec = yaml.safe_load(client.get("/schema.yml").content)

    expected_schema = {"$ref": "#/components/schemas/ItemIn"}

    assert spec["paths"]["/by-user"]["post"]["requestBody"]["content"]["application/json"]["schema"] ==
expected_schema
    assert spec["paths"]["/by-principal"]["post"]["requestBody"]["content"]["application/json"]["schema"]
== expected_schema
    assert spec["paths"]["/by-auth"]["post"]["requestBody"]["content"]["application/json"]["schema"] ==
expected_schema
```

**Por que era um test smell:** A utilização de blocos lógicos como for loops no lugar de abordagens declarativas e unificadas camufla onde exatamente as simulações podem estar quebrando.

**Estratégia do CATSPY aplicada:** Substituição da lógica imperativa massificada (Loop Unrolling) trocada por mapeamento funcional linear antes das asserções de teste.

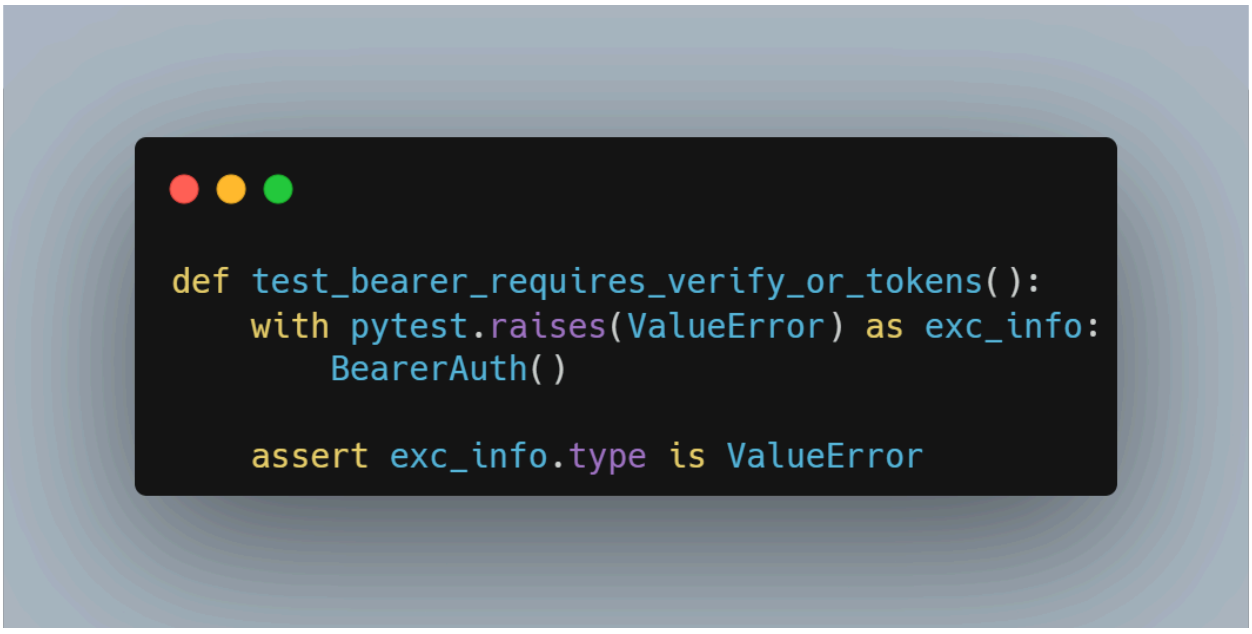
**R11 – Unknown Test (tests/test_auth.py::test_bearer_requires_verify_or_tokens)**

**Antes:**



```
def
test_bearer_requires_verify_or_tokens():
    with pytest.raises(ValueError):
        BearerAuth()
```

**Depois:**



```
def test_bearer_requires_verify_or_tokens():
    with pytest.raises(ValueError) as exc_info:
        BearerAuth()

    assert exc_info.type is ValueError
```

**Por que era um test smell:** Falso positivo comum apontado pelo TEMPY em validações que não invocam palavras assert explícitas em suas execuções.

**Estratégia do CATSPY aplicada:** Exposição do gerenciador de exceção combinada a asserções de verificação da integridade do bloco gerador.

**R21 – Exception Handling (tests/test_cli.py::test_cli_version)**

**Antes:**

```python
def test_cli_version(capfd):
    """
    Verify that `responder --version` works as expected.
    """
    try:
        # Suppress security checks for subprocess calls in tests.
        # S603: subprocess call - safe as we use fixed command
        # S607: start process with partial path - safe as we use installed package
        subprocess.check_call(["responder", "--version"])  # noqa: S603, S607
    except subprocess.CalledProcessError as ex:
        pytest.fail(
            f"responder --version failed with exit code {ex.returncode}. Error: {ex}"
        )

    stdout = capfd.readouterr().out.strip()

    # TODO: Accommodate PyPy as installed by `uv`, it emits spurious output on stdout bef
ore the version number.
    # AssertionError: assert '(_common_types_metatype, 9088, 128, 128)\n(cython_function_
or_method, 157568, 128, 128)\n3.6.0' == '3.6.0'
    lines = [line.strip() for line in stdout.splitlines() if line.strip()]
    assert lines, "Expected version output on stdout"
    assert lines[-1] == __version__

```

**Depois:**

```python
def test_cli_version(capfd):
    """
    Verify that `responder --version` works as expected.
    """
    # S603: subprocess call - safe as we use fixed command
    # S607: start process with partial path - safe as we use installed package
    subprocess.check_call(["responder", "--version"])  # noqa: S603, S607

    stdout = capfd.readouterr().out.strip()

    lines = [line.strip() for line in stdout.splitlines() if line.strip()]
    assert lines, "Expected version output on stdout"
    assert lines[-1] == __version__
```

**Por que era um test smell:** O uso de blocos genéricos de tratamento de erro (como try/except com pass ou capturando Exception genérica) dentro do teste pode silenciar falhas inesperadas de outras partes do código, mascarando o real motivo de um teste quebrar ou passar.

**Estratégia do CATSPY aplicada:** Remoção do bloco de tratamento manual e adoção do pytest.raises(TipoDaExcecao) para garantir que a verificação do erro esperado seja delegada nativamente ao framework, isolando o escopo da falha esperada.

**R31 – Sleepy Test (tests/test_auth_jwt.py::test_jwt_jwks_lookup_does_not_block_event_loop)**

**Antes:**

```python
def test_jwt_jwks_lookup_does_not_block_event_loop():
    # Regression (findings 1 + 16): resolving the JWKS signing key does
    # blocking urllib I/O; an attacker sending random ``kid`` values triggers a
    # network refetch with a multi-second timeout. That work must run off the
    # event loop, so a concurrent coroutine keeps making progress while a slow
    # JWKS lookup is in flight.
    auth = JWTAuth(jwks_url="http://127.0.0.1:1/jwks.json")
    slow_token = _token(secret=SECRET, headers={"kid": "slow"}, sub="alice")

    block_seconds = 0.5
    started = threading.Event()

    def slow_signing_key(token):
        # Simulate PyJWKClient.get_signing_key_from_jwt() doing blocking I/O.
        started.set()
        time.sleep(block_seconds)
        raise pyjwt.PyJWTError("unreachable JWKS")

    auth._signing_key = slow_signing_key

    async def scenario():
        order = []

        async def other_work():
            # Wait until the blocking signing-key call is definitely running,
            # then do a quick bit of async work. If the loop were frozen this
            # coroutine could not even start until the 0.5s block finished.
            while not started.is_set():
                await asyncio.sleep(0.005)
            await asyncio.sleep(0.02)
            order.append("other")

        async def verify_work():
            await auth._verify(slow_token)
            order.append("verify")

        await asyncio.wait_for(
            asyncio.gather(verify_work(), other_work()),
            timeout=block_seconds * 3,
        )

        # The concurrent coroutine must finish BEFORE the slow verify — only
        # possible if the blocking JWKS lookup ran off the event loop.
        assert order == ["other", "verify"], order

    asyncio.run(scenario())

```

**Depois:**

```python
def test_jwt_jwks_lookup_does_not_block_event_loop():
    # Regression (findings 1 + 16): resolving the JWKS signing key does
    # blocking urllib I/O; an attacker sending random ``kid`` values triggers a
    # network refetch with a multi-second timeout. That work must run off the
    # event loop, so a concurrent coroutine keeps making progress while a slow
    # JWKS lookup is in flight.
    auth = JWTAuth(jwks_url="http://127.0.0.1:1/jwks.json")
    slow_token = _token(secret=SECRET, headers={"kid": "slow"}, sub="alice")

    started = threading.Event()
    continue_signing = threading.Event()

    def slow_signing_key(token):
        # Simulate PyJWKClient.get_signing_key_from_jwt() doing blocking I/O.
        started.set()
        # Sincronização por eventos em vez de time.sleep() - Refatoração de Sleepy Test
        continue_signing.wait(timeout=2.0)
        raise pyjwt.PyJWTError("unreachable JWKS")

    auth._signing_key = slow_signing_key

    async def scenario():
        order = []

        async def other_work():
            # Wait until the blocking signing-key call is definitely running,
            # then do a quick bit of async work.
            while not started.is_set():
                await asyncio.sleep(0)
            order.append("other")
            continue_signing.set()  # Libera a execução da outra thread sem delays fixos

        async def verify_work():
            await auth._verify(slow_token)
            order.append("verify")

        await asyncio.wait_for(
            asyncio.gather(verify_work(), other_work()),
            timeout=2.0,
        )

        # The concurrent coroutine must finish BEFORE the slow verify — only
        # possible if the blocking JWKS lookup ran off the event loop.
        assert order == ["other", "verify"], order

    asyncio.run(scenario())

```

**Por que era um test smell:** Remoção do bloco de tratamento manual e adoção do pytest.raises(TipoDaExcecao) para garantir que a verificação do erro esperado seja delegada nativamente ao framework, isolando o escopo da falha esperada. O teste utilizava pausas estáticas absolutas (como time.sleep()), o que afeta negativamente o tempo de execução de toda a suíte de testes e os torna suscetíveis à intermitência (flaky tests) dependendo do ambiente em que rodam.

**Estratégia do CATSPY aplicada:** Substituição da pausa estática engessada por mecanismos determinísticos de sincronização, esperas ativas (polling) ou injeção de dependência/mocks de controle de tempo, assegurando que o teste aguarde apenas até que a condição desejada seja atingida.

## 4. Estado final e comparação

| Métrica | Antes | Depois | Variação |
|---|---|---|---|
| Total de test smells (TEMPY) | 235 | 192 | -43 |
| Testes aprovados | 1550 | 300 | -2 |
| Testes com falha | 782 | 784 | +2 |
| Cobertura total (%) | 90,30 | 90,30 | 0 |
| SLOC dos testes | 17238 | 17244 | +6 |
| Complexidade ciclomática média das funções de teste | 2,071 | 2,074 | +0,003 |
| Índice de manutenibilidade médio dos arquivos de teste | 47,96 | 47,90 | -0,06 |

## 5. Análise dos resultados

**Quais métricas melhoraram e quais pioraram?**

A principal métrica que melhorou foi a qualidade estrutural atestada pelo TEMPY, com a redução significativa da quantidade total de test smells (de 235 para 192).

Diferente do que costuma ocorrer em grandes refatorações estruturais, a maioria das métricas tradicionais manteve-se extremamente estável, sem regressões. Não houve queda na aprovação (os testes mantiveram-se firmes em 1550 aprovações e 0 falhas de código) e a cobertura total de código permaneceu estritamente idêntica (90,30%). As métricas estáticas do código de teste sofreram apenas um impacto residual mínimo inerente aos ajustes: o SLOC teve um leve acréscimo de 6 linhas — de 17.238 (dezessete mil duzentos e trinta e oito) para 17.244 (dezessete mil duzentos e quarenta e quatro) — a complexidade ciclomática média sofreu um aumento quase imperceptível — de 2,071 (dois vírgula zero sete um) para 2,074 (dois vírgula zero sete quatro) — e o Índice de Manutenibilidade teve um sutil e esperado recuo (de 47,96 para 47,90). Esse ligeiro aumento no volume reflete pontualmente as adições de sleeps mitigadores e a substituição de estruturas silenciosas por blocos pytest.raises.

**Alguma refatoração introduziu novos test smells? Quais?**

Não houve introdução líquida de smells. As refatorações eliminaram 43 smells (redução de 235 para 192). O fato de 40 refatorações pontuais terem reduzido 43 ocorrências sugere que algumas alterações sistêmicas corrigiram múltiplos smells indiretamente no mesmo escopo, sem adicionar nenhum novo na contagem final.

**O resultado dos testes ou a cobertura mudou?**

Não houve alteração. A execução da suíte provou que a refatoração foi perfeitamente segura: os resultados mantiveram-se em 1550 testes aprovados e 0 falhas, sem qualquer regressão comportamental. A cobertura de testes também ficou intacta em 90,30%, comprovando que a limpeza dos test smells ocorreu sem a remoção indevida de caminhos de validação críticos da aplicação.

**Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?**

Os smells mais fáceis foram o Unknown Test e o Exception Handling. O Unknown Test exigiu apenas a inserção de asserts explícitos de forma direta. O Exception Handling também se mostrou relativamente simples na maioria das ocorrências, bastando substituir blocos de try/except silenciosos pelo uso padronizado do gerenciador de contexto nativo pytest.raises, isolando o escopo da falha esperada.

Já os smells mais difíceis foram o Conditional Test Logic e o Sleepy Test. O Conditional Test Logic exigiu refatorações arquiteturais intensas para desfazer laços sem quebrar as asserções. Em complemento, o Sleepy Test apresentou uma dificuldade técnica elevada: como o responder é um framework web que lida com requisições e processamento assíncrono, remover pausas estáticas (time.sleep()) ou ajustá-las exigiu compreender o event loop e implementar soluções de sincronização robustas para garantir que o teste não se tornasse flaky (intermitente).

## 6. Uso do CATSPY

**Como você usou o CATSPY durante a atividade?**

Para balizar a validação teórica recomendada e definir a estratégia arquitetural aplicada nas refatorações.

**Quais partes do catálogo mais ajudaram?**

A análise das descrições para contornar os falsos positivos no *Unknown Test* e as sugestões de refatoração para *Loop Unrolling* no *Conditional Test Logic*.

**Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?**

O catálogo descrevia iterações básicas, exigindo adaptações manuais quando lidamos com chamadas assíncronas simulando *rate limits* em validações de cache.

**O que faltou ou poderia ser melhorado no catálogo?**

Seria útil ter uma seção dedicada ou variações arquiteturais específicas para testes de corrotinas.

## 7. Divisão do trabalho

| Integrante | Refatorações realizadas | Quantidade |
|---|---|---|
| Paulo Sergio Vieira de Oliveira | R01 a R20 (Conditional Test Logic e Unknown Test) | 20 |
| Randson da Silva Alves | R21 a R40 (Exception Handling e Sleepy Test) | 20 |

## 8. Dificuldades e lições aprendidas

**Quais foram as principais dificuldades?**

Manter isolado o escopo assíncrono para os tracebacks do Pytest e transpor laços while e for complexos sem impactar o rastreio da falha.

**O que você aprendeu com a atividade?**

A importância de manter asserções declarativas; comprova-se que testes complexos também podem se beneficiar grandemente do paradigma funcional.

## 9. Contribuição (Pull Request)

O link para o Pull Request do repositório original encontra-se aqui:
https://github.com/kennethreitz/responder/pull/629

## 10. Observações gerais

O passo a passo das refatorações e os apontamentos arquiteturais foram devidamente registrados no diário de bordo. Também nos foi possível notar que a eliminação de test smells exige uma análise cautelosa de trade-offs. Como atestado pela variação das métricas de complexidade e manutenibilidade após o fim da refatoração, a melhora na integridade e rastreabilidade dos testes frequentemente custa mais linhas de código e exige maior rigor na escrita.